Tutorial 1B (to be completed in week1)
Data Modeling (Part 1)

Tutorial Objectives:

- Introduce JDeveloper (Oracle Client)
- Draw a conceptual model using JDeveloper

Oracle JDeveloper is a free integrated development environment that simplifies the development of applications with specific features that address the various technologies such as Java, Database, etc. We will use JDeveloper in relation to the Database technology. Access Oracle's JDeveloper via VMWare Horizon.

Notes:

As mentioned earlier in Tutorial 1A, JDeveloper is used mainly as a drawing tool (e.g., draw conceptual model) providing an alternative to Visio.

1. JDeveloper (version 12.2.1.4.0) is available in the Massey Computer labs and the tutorial instructions are based on the same version.

Tutorial Tasks:

a. Create a conceptual model for the given case study using an ERD
b. Represent the ERD as a UML class diagram
c. Extend the class diagram (add attributes, etc.)

Part A

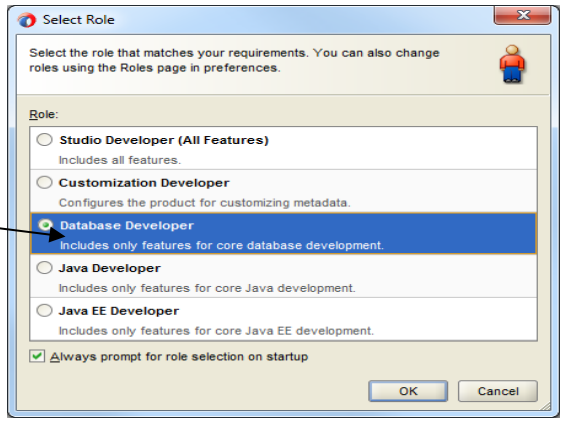
a. Create a Conceptual model for the given case study using an ERD

Read the Tiny College case study* on pg. 57-58, ch2 (Ed13) to identify the main entities and relationships.
Represent these entities and relationships as an ERD (hand-drawn is fine at this stage).

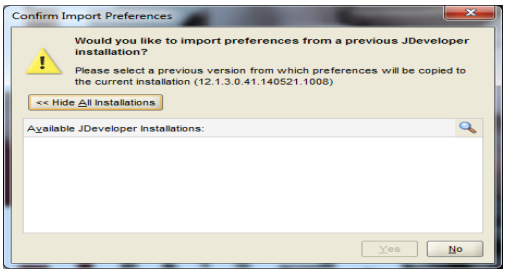
Part B

b. Represent the ERD as a class diagram (using Oracle JDeveloper)

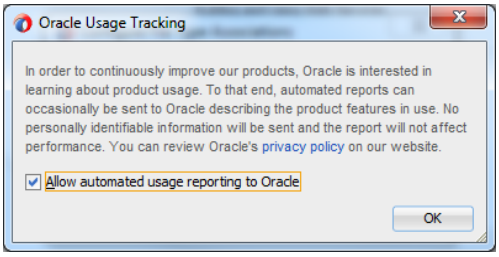
i. Load JDeveloper (START→Oracle-OraClient12Home→Oracle JDeveloper Studio; alternatively search the application) and choose the Database Developer under Select Role.



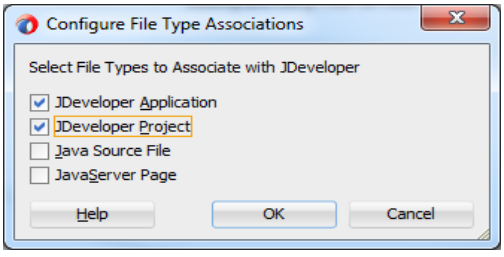
ii. This does not apply unless you have previous JDeveloper installation.



iii. Close the Oracle Usage Tracking prompt (and Tip of the Day window some versions show this). The JDeveloper IDE should be displayed.



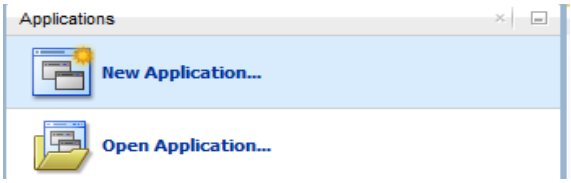
iv. Select JDeveloper Application and JDeveloper Project file types to associate with JDeveloper.

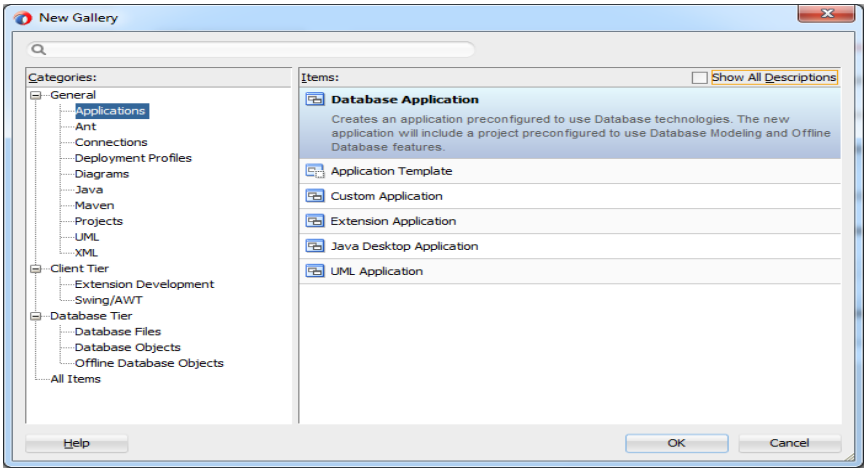


Notes: When you work in JDeveloper, you organise your work in ***Projects*** that reside under ***Applications***. This tutorial guides you through creating an application and a project without using any predefined template; you later customise the project to enable you to utilise JDeveloper's database functionality.

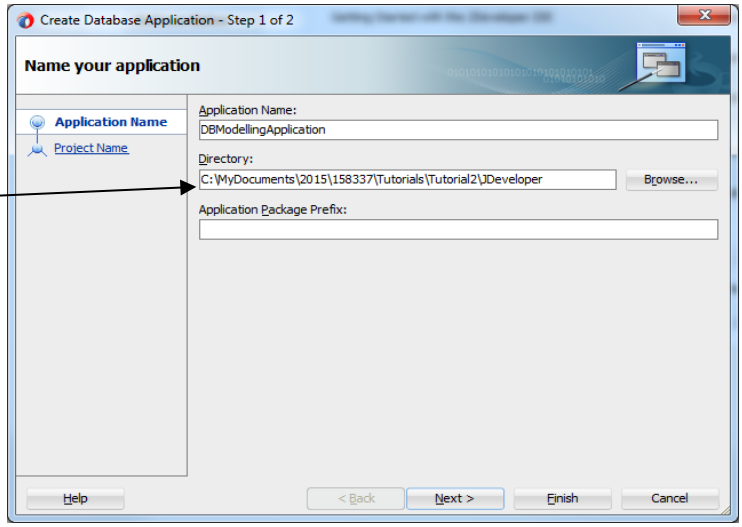
v. Creating the Model Project

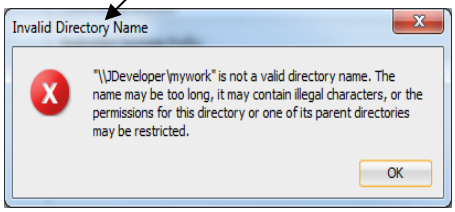
i. In the Applications (Navigator), click the New Application link. New Gallery screen will show. Select Database Application.





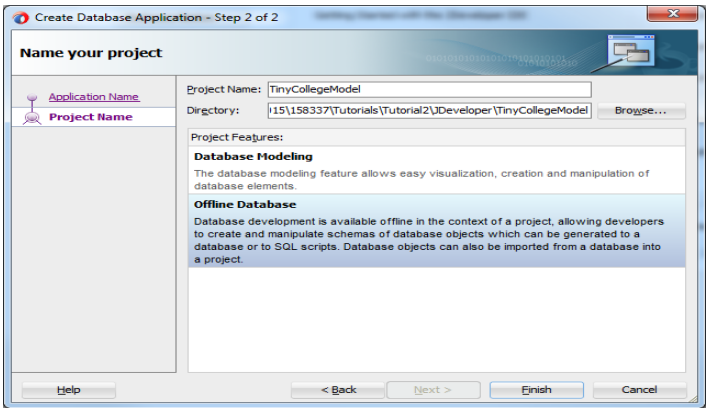
ii. Change Application Name from the default Application1 (e.g. DBModelingApplication). Use Browse button to change the Directory from the default location (\\JDeveloper\mywork) to say C:\I\ (if temporarily saving on Desktop) or any other location otherwise you will get the Invalid Directory Name dialog box on clicking Next.





To access your application later, first open the JDeveloper and then use Open Application.

iii. Enter Project Name (e.g. TinyCollegeModel) and under the Project Features, select Offline Database, then Click Finish.

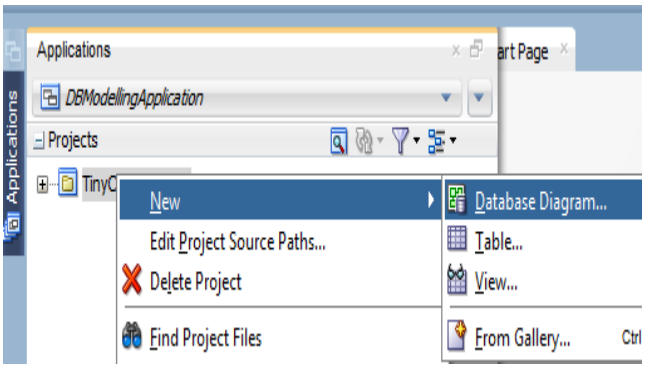


Save your work by clicking Save All. You should save your work at regular intervals.

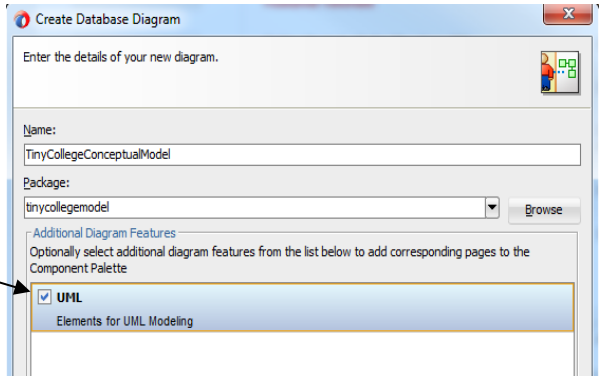
vi. Using a UML Class Model to do (Conceptual) Database Modeling

i. Prepare a Class Model Diagram Environment by performing following steps:

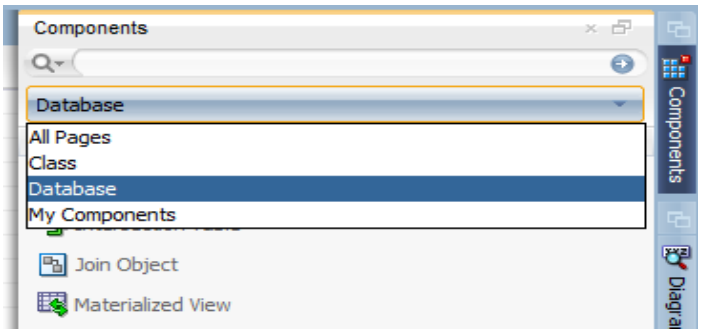
Right-click the (**TinyCollege**) **Model** project in the Applications (Navigator) and choose **New** →Database Diagram from the context menu.

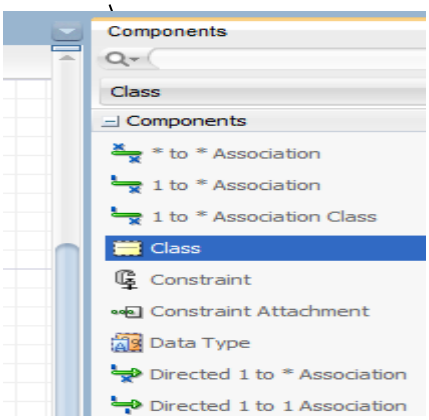


In the Create Database Diagram window, enter the details of your new diagram. Name the diagram (e.g. TinyCollegeConceptualModel) and check the UML checkbox to include the UML elements (e.g. class diagram).
A new database diagram opens for you to draw.

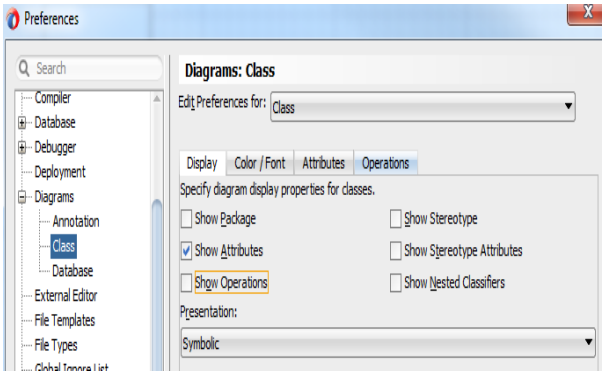


Under Components (Navigator), select Class, as we need to create a Class diagram. The Component Palette provides the elements (e.g. class, association, etc.) that you can use on this diagram.

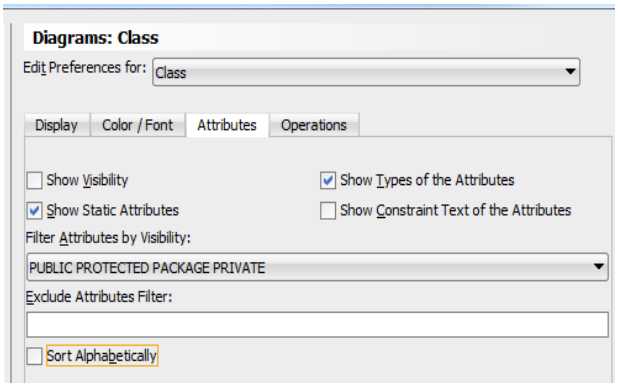




Select **Tools -> Preferences** to set some class diagram preferences. In the Preferences dialog, select **Diagrams -> Class**, and in the **Edit Preferences for**, choose **Class**, and uncheck **Show Operations** as we are doing DB modeling and do not need operations. You might want to explore other options under different tabs (e.g., Attributes, Operations, etc.).



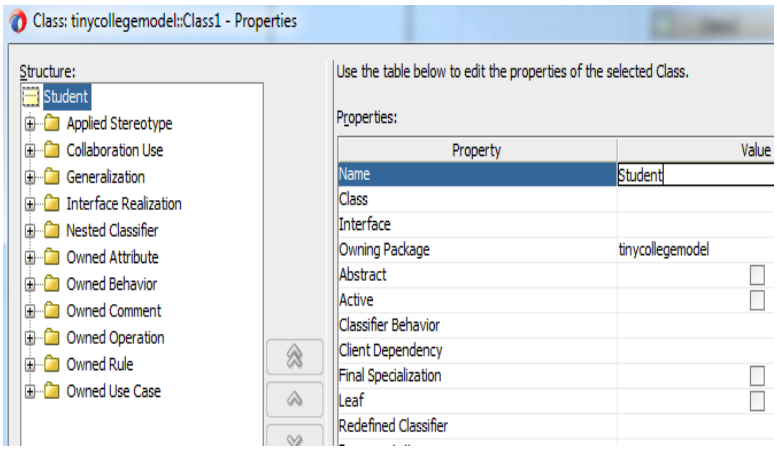
Click the **Attributes** tab and uncheck **Show Visibility** as that is not needed. Turn off **Sort Alphabetically** if you do not wish to have your attributes sorted in alphabetical order. You can play with other properties as well. These preferences will be set for all Class diagrams.

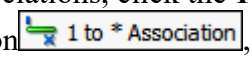


ii. Creating a Class Model Diagram

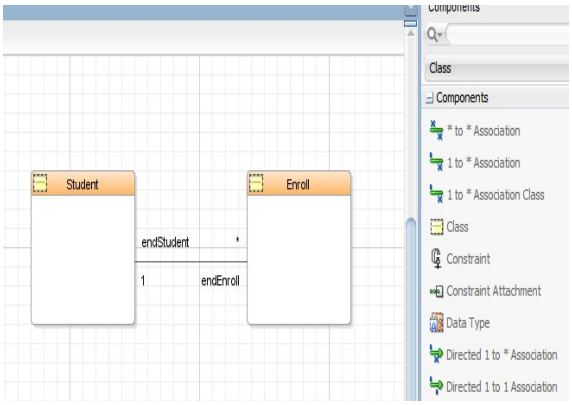
In the Component Palette, select the **Class** component, 

and drag and drop it onto the diagram. Change the class name (e.g. Student) either directly on the diagram or by accessing the class property. Similarly, create other classes (e.g. Enroll, Class, Course, etc.).

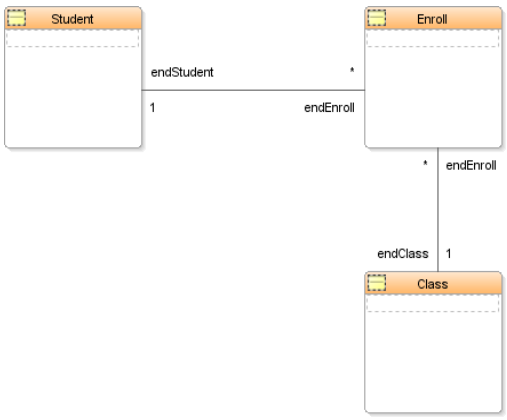


For adding associations, click the **1 to * Association** icon 

, then click within the **Student** class and draw a line, click within the **Enroll** class. Add all the required associations. Double click the **association** to open the Properties (e.g. association name, etc.) but at this stage, you can ignore most modifications.



The diagram shows only a small section of the class diagram.



PART c

c. Extend the class diagram to meet case requirements (e.g. add attributes)

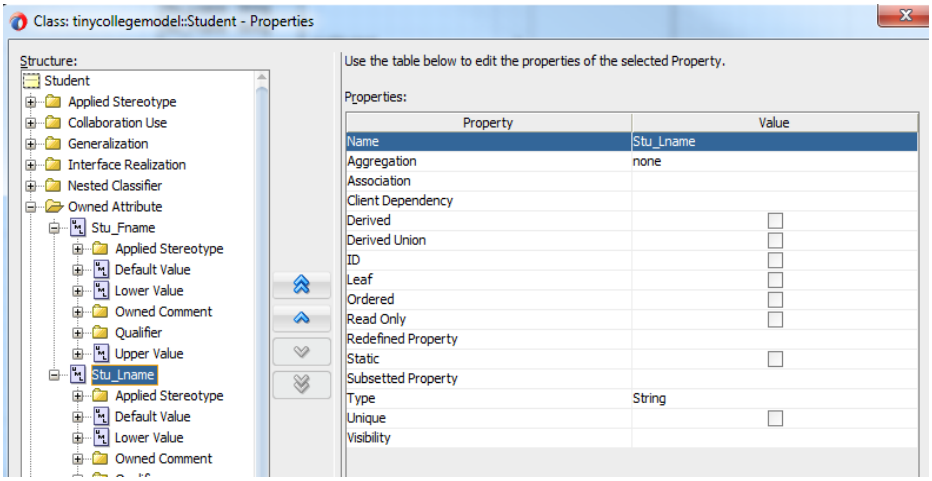
i. Using in-place edit, add some of the known attributes to classes (e.g. student first name, course title, etc.). However, **do not** include ids for the classes (e.g. crs_id, stu_num) as these are automatically generated (and flagged as primary keys) later by the CASE tool when converting the conceptual model to the logical data model.

ii. In the **Student** class, click once to select the place (goes blue) click again to enter edit mode (don't double click) and over-write in format **<att name>:<DB type> Stu_Fname : String.**



Alternatively, Open the Properties window for the Class (e.g., Student) by double clicking and add the New Attribute (e.g., Stu_Lname) under Owned Attribute.

iii. Consider changing some properties (e.g. Type – String, uncheck Unique as there can be many students with the same Last name). You can even move the attributes up and down.

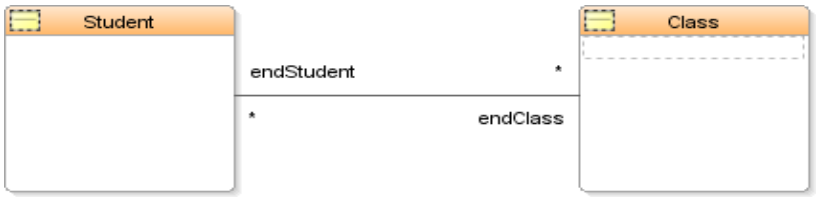


iv. 'Print Preview' your diagram and if necessary consider various Page Setup (Orientation, Margins, etc.) and Print options (Header-Footer, Fit to Page, etc.) before printing.

Notes:

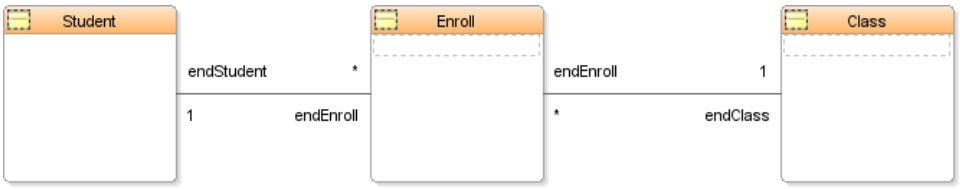
1. Confirm (either with the client or in client's absence list under assumptions) the ambiguous business rules (these rules translate to multiplicities) before you put them on the diagram. Thinking on the lines of following situations is helpful.
   - What if a Professor is on sabbatical leave or is new and therefore, not yet assigned to any Class?
   - What if a Class is taught by more than one Professor?
   - Are we interested in storing data about non-teaching rooms as well?
   - Do we want to allow students to be part of the database even if they have not enrolled for any Course(s) yet?

2. Although a conceptual diagram (technology and implementation independent) may have many-to-many (M:N) relationships but we encourage you to think about the intersection entities right from the start (relational models do not support M:M) and therefore resolve any M:N relationships by having two 1:M relationships. For example, for a M:N relationship between the entities Student and Class, think about an intersection entity (e.g. Enroll).



Avoid M:N relationship between the classes Student and Class shown above;

The diagram below now includes an intersection entity Enroll between the Student and Class entities and has two 1:M relationships.



***Screen Dump of the Tiny College Case Study (pg 57-58)**

- A PROFESSOR may teach many CLASSes, and each CLASS is taught by only one PROFESSOR; there is a 1:M relationship between PROFESSOR and CLASS.
- A CLASS may ENROLL many students, and each STUDENT may ENROLL in many CLASSes, thus creating an M:N relationship between STUDENT and CLASS. (You will learn about the precise nature of the ENROLL entity in Chapter 4.)
- Each COURSE may generate many CLASSes, but each CLASS references a single COURSE. For example, there may be several classes (sections) of a database course that have a course code of CIS-420. One of those classes might be offered on MWF from 8:00 a.m. to 8:50 a.m., another might be offered on MWF from 1:00 p.m. to 1:50 p.m., while a third might be offered on Thursdays from 6:00 p.m. to 8:40 p.m. Yet, all three classes have the course code CIS-420.
- Finally, a CLASS requires one ROOM, but a ROOM may be scheduled for many CLASSes. That is, each classroom may be used for several classes: one at 9:00 a.m., one at 11:00 a.m., and one at 1:00 p.m., for example. In other words, there is a 1:M relationship between ROOM and CLASS.